

AI ASS 9.1

Name: B.Sanjana

H.T.NO: 2303A52306

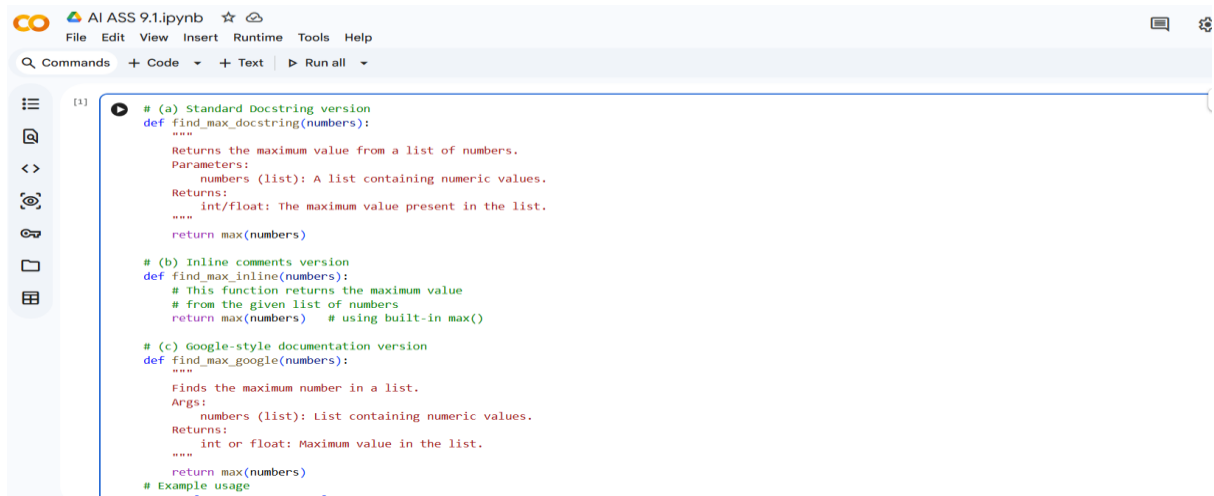
Batch: 43

Problem 1:

Consider a Python function `find_max(numbers)` that returns the maximum value from a list. Write documentation for this function using **docstring**, **inline comments**, and **Google-style documentation**.

Compare the three documentation styles by discussing their advantages, disadvantages, and suitable use cases.

Finally, recommend the most appropriate documentation style for a **mathematical utilities library** with justification.

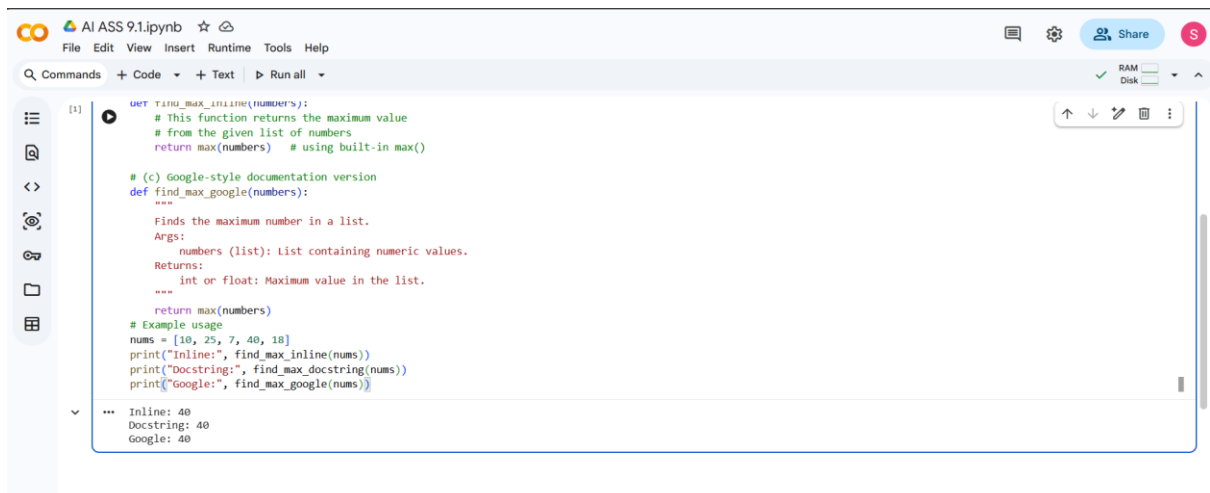


```
[1] In [ ]: # (a) Standard Docstring version
def find_max_docstring(numbers):
    """
    Returns the maximum value from a list of numbers.
    Parameters:
        numbers (list): A list containing numeric values.
    Returns:
        int/float: The maximum value present in the list.
    """
    return max(numbers)

# (b) Inline comments version
def find_max_inline(numbers):
    """
    This function returns the maximum value
    from the given list of numbers
    """
    return max(numbers) # using built-in max()

# (c) Google-style documentation version
def find_max_google(numbers):
    """
    Finds the maximum number in a list.
    Args:
        numbers (list): List containing numeric values.
    Returns:
        int or float: Maximum value in the list.
    """
    return max(numbers)

# Example usage
nums = [10, 25, 7, 40, 18]
```



```
[1] In [ ]: def find_max_docstring(numbers):
    """
    Returns the maximum value from a list of numbers.
    Parameters:
        numbers (list): A list containing numeric values.
    Returns:
        int/float: The maximum value present in the list.
    """
    return max(numbers)

def find_max_inline(numbers):
    """
    This function returns the maximum value
    from the given list of numbers
    """
    return max(numbers) # using built-in max()

def find_max_google(numbers):
    """
    Finds the maximum number in a list.
    Args:
        numbers (list): List containing numeric values.
    Returns:
        int or float: Maximum value in the list.
    """
    return max(numbers)

# Example usage
nums = [10, 25, 7, 40, 18]
print("Inline:", find_max_inline(nums))
print("Docstring:", find_max_docstring(nums))
print("Google:", find_max_google(nums))

*** Inline: 40
Docstring: 40
Google: 40
```

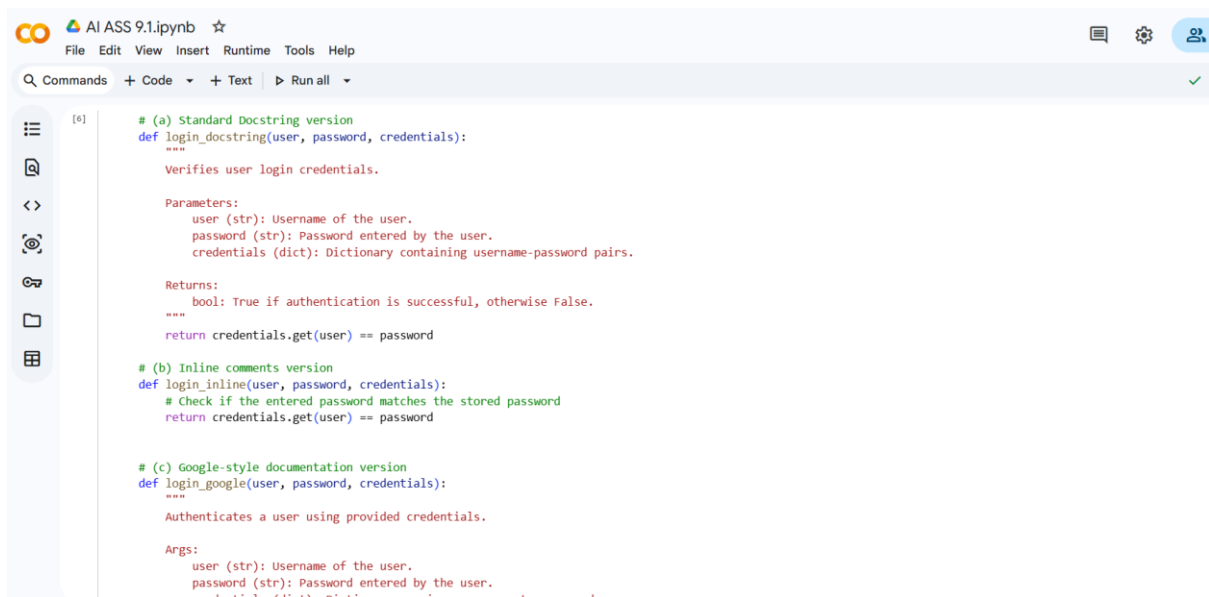
Problem 2:

Consider the Python function `login(user, password, credentials)` that checks whether a username and password match a stored dictionary of credentials.

Write documentation for this function in three formats: inline comments, standard docstring, and Google-style documentation.

Compare the three approaches by discussing their advantages, disadvantages, and suitable use cases.

Finally, recommend which documentation style is most effective for helping new developers onboard a project, and justify your choice.



```
AI ASS 9.1.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all

# (a) Standard Docstring version
def login_docstring(user, password, credentials):
    """
    Verifies user login credentials.

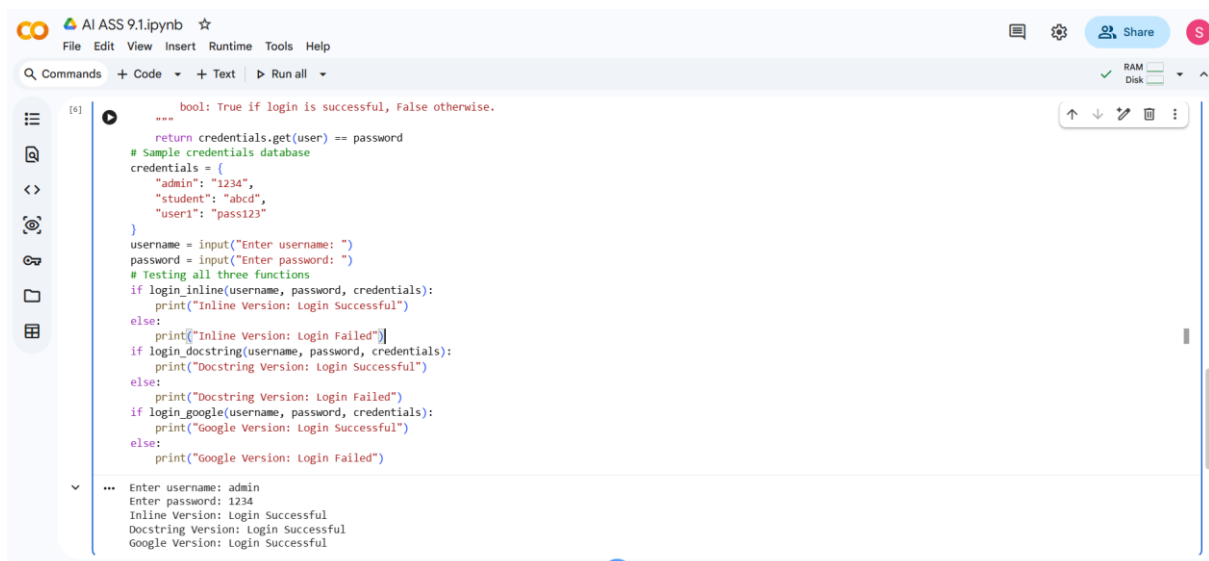
    Parameters:
        user (str): Username of the user.
        password (str): Password entered by the user.
        credentials (dict): Dictionary containing username-password pairs.

    Returns:
        bool: True if authentication is successful, otherwise False.
    """
    return credentials.get(user) == password

# (b) Inline comments version
def login_inline(user, password, credentials):
    # Check if the entered password matches the stored password
    return credentials.get(user) == password

# (c) Google-style documentation version
def login_google(user, password, credentials):
    """
    Authenticates a user using provided credentials.

    Args:
        user (str): Username of the user.
        password (str): Password entered by the user.
        credentials (dict): Dictionary mapping username to password
    """
```



```
AI ASS 9.1.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all
Share RAM Disk

bool: True if login is successful, False otherwise.
"""
return credentials.get(user) == password
# Sample credentials database
credentials = {
    "admin": "1234",
    "student": "abcd",
    "user1": "pass123"
}
username = input("Enter username: ")
password = input("Enter password: ")
# Testing all three functions
if login_inline(username, password, credentials):
    print("Inline Version: Login Successful")
else:
    print("Inline Version: Login Failed")
if login_docstring(username, password, credentials):
    print("Docstring Version: Login Successful")
else:
    print("Docstring Version: Login Failed")
if login_google(username, password, credentials):
    print("Google Version: Login Successful")
else:
    print("Google Version: Login Failed")

... Enter username: admin
Enter password: 1234
Inline Version: Login Successful
Docstring Version: Login Successful
Google Version: Login Successful
```

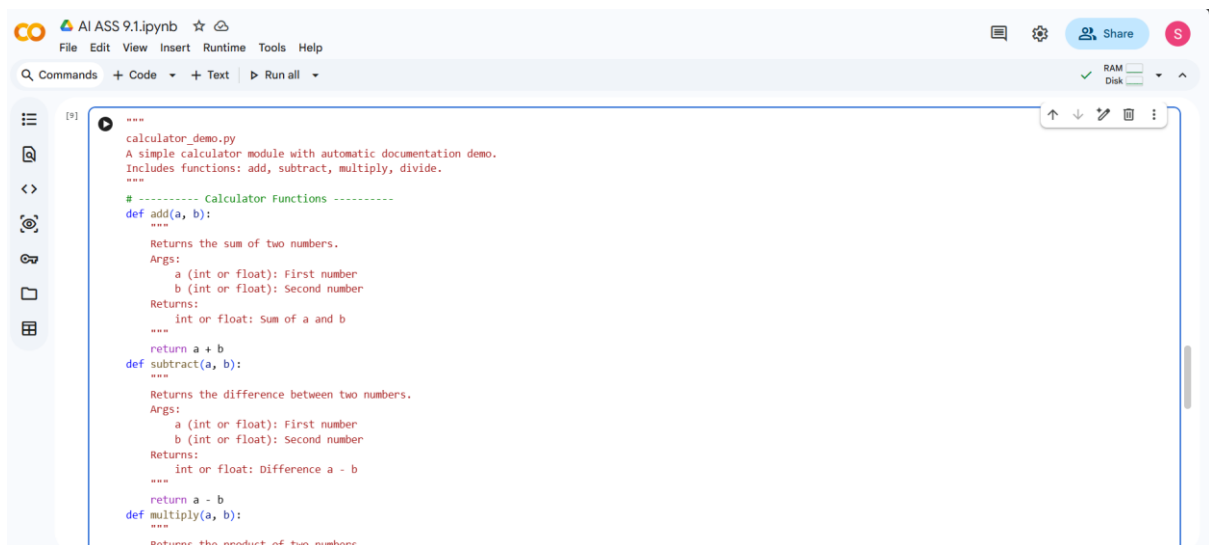
Problem 3:

Design a Python module named `calculator.py` that provides basic arithmetic functions: `add(a, b)`, `subtract(a, b)`, `multiply(a, b)`, and `divide(a, b)`.

Write appropriate docstrings for each function.

Demonstrate automatic documentation generation by displaying the module and function docstrings in the terminal using Python's documentation tools.

Also, generate and export the module documentation in HTML format using `pydoc` and verify it in a web browser.



```
calculator_demo.py
A simple calculator module with automatic documentation demo.
Includes functions: add, subtract, multiply, divide.
# ----- Calculator Functions -----
def add(a, b):
    """
    Returns the sum of two numbers.
    Args:
        a (int or float): First number
        b (int or float): Second number
    Returns:
        int or float: Sum of a and b
    """
    return a + b
def subtract(a, b):
    """
    Returns the difference between two numbers.
    Args:
        a (int or float): First number
        b (int or float): Second number
    Returns:
        int or float: Difference a - b
    """
    return a - b
def multiply(a, b):
    """
    Returns the product of two numbers.
```



```
    return a * b
def divide(a, b):
    """
    Returns the quotient of two numbers.
    Args:
        a (int or float): Dividend
        b (int or float): Divisor
    Returns:
        float: Result of a / b
    Raises:
        ValueError: If b is zero
    """
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b
# ----- Demonstration & Documentation -----
if __name__ == "__main__":
    print("==== Calculator Demo =====")
    # Example usage
    x, y = 10, 5
```

The screenshot shows a Jupyter Notebook environment with a file named 'AI ASS 9.1.ipynb'. The code in the notebook defines a calculator module with the following structure:

```
""" Calculator Module """
Returns:
    float: Result of a / b
Raises:
    ValueError: If b is zero
"""
if b == 0:
    raise ValueError("Cannot divide by zero.")
return a / b

# ----- Demonstration & Documentation -----
if __name__ == "__main__":
    print("==== Calculator Demo =====")
    # Example usage
    x, y = 10, 5
    print(f"{x} + {y} = {add(x, y)}")
    print(f"{x} - {y} = {subtract(x, y)}")
    print(f"{x} * {y} = {multiply(x, y)}")
    print(f"{x} / {y} = {divide(x, y)}\n")
```

Below the code, the output of the demo is shown:

```
==== Calculator Demo =====
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.0
```

Problem 4:

1. Create a Python module named `conversion.py` that includes the following functions:

- `decimal\_to\_binary(n)` : Converts a decimal number `n` to its binary string representation.

- `binary\_to\_decimal(b)` : Converts a binary string `b` to its decimal number.

- `decimal\_to\_hexadecimal(n)` : Converts a decimal number `n` to its hexadecimal string representation.

2. Add proper docstrings for each function (use Copilot or AI for auto-generation if needed).

3. Generate documentation for the module in the terminal using Python's built-in tools.

4. Export the documentation to HTML format and open it in a browser.

Extra:

- Include a test block that demonstrates all three conversions.

- Ensure the script is self-contained, so running it executes the tests and opens the documentation.

AI ASS 9.1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[10]

```
# conversion_with_docs.py

import pydoc
import webbrowser
import os

def decimal_to_binary(n: int) -> str:
    """
    Convert a decimal number to its binary representation.
    Parameters:
    n (int): The decimal number to convert.
    Returns:
    str: Binary representation of the decimal number.
    """
    return bin(n)[2:] # Remove '0b' prefix

def binary_to_decimal(b: str) -> int:
    """
    Convert a binary number (string) to its decimal equivalent.
    Parameters:
    b (str): Binary number as a string.
    Returns:
    int: Decimal equivalent of the binary number.
    """
    return int(b, 2)

def decimal_to_hexadecimal(n: int) -> str:
    """
    Convert a decimal number to its hexadecimal representation.
    Parameters:
    n (int): The decimal number to convert
```

RAM
Disk

AI ASS 9.1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[10]

```
Convert a decimal number to its hexadecimal representation.
Parameters:
n (int): The decimal number to convert.
Returns:
str: Hexadecimal representation of the decimal number.
"""
return hex(n)[2:] # Remove '0x' prefix

# Generate HTML docs & open
def generate_docs_and_open():
    """
    Generate HTML documentation for this module and open it in a browser.
    """
    # Create HTML doc using pydoc
    filename = "conversion.html"
    pydoc.writedoc(__name__)
    html_file = __name__ + ".html"
    if os.path.exists(html_file):
        os.rename(html_file, filename)
    # Open HTML in default web browser
    webbrowser.open(filename)
    print(f"Documentation generated and opened in browser: {filename}")

# Test functions
def test_conversions():
    print("Decimal 10 to binary:", decimal_to_binary(10))
    print("Binary '1010' to decimal:", binary_to_decimal('1010'))
    print("Decimal 255 to hexadecimal:", decimal_to_hexadecimal(255))

if __name__ == "__main__":
    test_conversions()
    generate_docs_and_open()
```

RAM
Disk

AI ASS 9.1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[10]

```
html_file = __name__ + ".html"
if os.path.exists(html_file):
    os.rename(html_file, filename)
# Open HTML in default web browser
webbrowser.open(filename)
print(f"Documentation generated and opened in browser: {filename}")

# Test functions
def test_conversions():
    print("Decimal 10 to binary:", decimal_to_binary(10))
    print("Binary '1010' to decimal:", binary_to_decimal('1010'))
    print("Decimal 255 to hexadecimal:", decimal_to_hexadecimal(255))

if __name__ == "__main__":
    test_conversions()
    generate_docs_and_open()
```

RAM
Disk

...

Decimal 10 to binary: 1010
Binary '1010' to decimal: 10
Decimal 255 to hexadecimal: ff
wrote __main__.html
Documentation generated and opened in browser: conversion.html

Problem 5:

1. Create a Python module named `course\_with\_docs.py` that includes the following functions:

- `add\_course(course\_id, name, credits)` : Adds a course to an internal course dictionary.

- `remove\_course(course\_id)` : Removes a course by its ID and returns True if removed, False if not found.

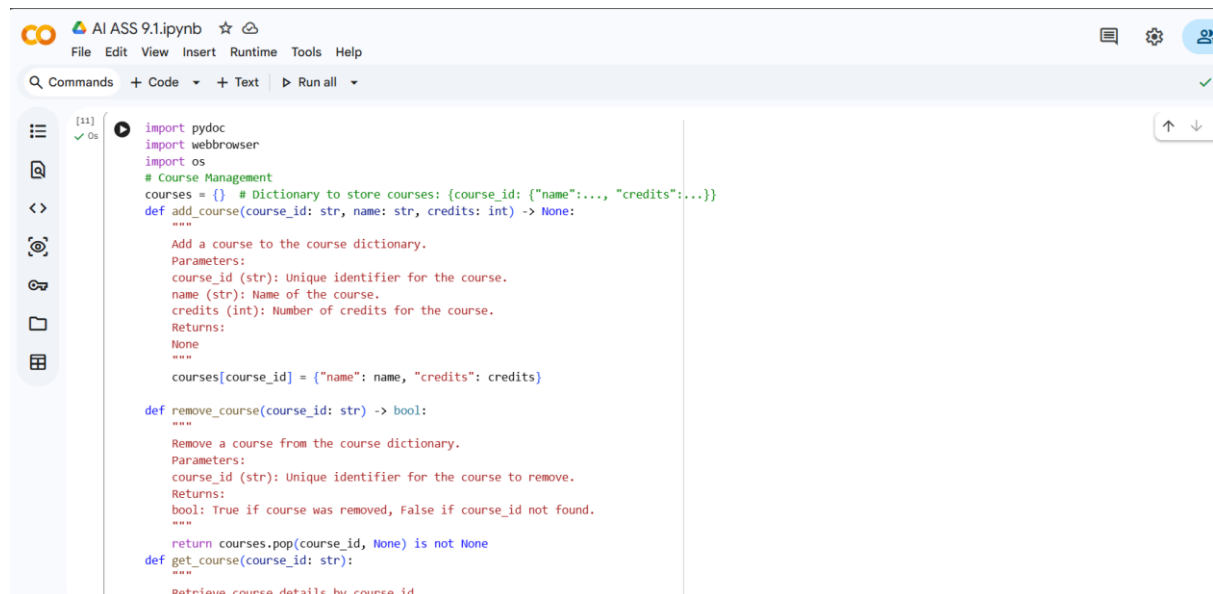
- `get\_course(course\_id)` : Returns the course details as a dictionary if found, else None.

2. Add proper docstrings for each function (use Copilot or AI for auto-generation).

3. Include a test function that demonstrates adding, retrieving, and removing courses.

4. Generate HTML documentation for the module using Python's built-in `pydoc`.

5. Automatically open the HTML documentation in the default web browser..



```
[11] ✓ Os
import pydoc
import webbrowser
import os

# Course Management
courses = {} # Dictionary to store courses: {course_id: {"name":..., "credits":...}}
def add_course(course_id: str, name: str, credits: int) -> None:
    """
    Add a course to the course dictionary.
    Parameters:
    course_id (str): Unique identifier for the course.
    name (str): Name of the course.
    credits (int): Number of credits for the course.
    Returns:
    None
    """
    courses[course_id] = {"name": name, "credits": credits}

def remove_course(course_id: str) -> bool:
    """
    Remove a course from the course dictionary.
    Parameters:
    course_id (str): Unique identifier for the course to remove.
    Returns:
    bool: True if course was removed, False if course_id not found.
    """
    return courses.pop(course_id, None) is not None
def get_course(course_id: str):
    """
    Retrieve course details by course_id.
    """
```

AI ASS 9.1.jpynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

111

0s

↑

↓

↻

🗑

```
course_id (str): Unique identifier for the course.
Returns:
dict or None: Dictionary with course details if found, else None.
"""

return courses.get(course_id)

# Generate HTML docs & open
def generate_docs_and_open():
    """
    Generate HTML documentation for this module and open it in a browser.
    """
    filename = "course.html"
    pydoc.writedoc(__name__)
    # pydoc writes 'course_with_docs.html', rename to simple 'course.html'
    html_file = __name__ + ".html"
    if os.path.exists(html_file):
        os.rename(html_file, filename)
    webbrowser.open(filename)
    print(f"Documentation generated and opened in browser: {filename}")

# Test functions
def test_courses():
    add_course("CSE101", "Data Structures", 4)
    add_course("CSE102", "Algorithms", 3)
    print("Get CSE101:", get_course("CSE101"))
    print("Remove CSE102:", remove_course("CSE102"))
    print("Get CSE102 after removal:", get_course("CSE102"))

# Main execution
if __name__ == "__main__":
    test_courses()
    generate_docs_and_open()
```

AI ASS 9.1.jpynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

111

0s

↑

↓

↻

🗑

⋮

```
"""
filename = "course.html"
pydoc.writedoc(__name__)
# pydoc writes 'course_with_docs.html', rename to simple 'course.html'
html_file = __name__ + ".html"
if os.path.exists(html_file):
    os.rename(html_file, filename)
webbrowser.open(filename)
print(f"Documentation generated and opened in browser: {filename}")

# Test functions
def test_courses():
    add_course("CSE101", "Data Structures", 4)
    add_course("CSE102", "Algorithms", 3)
    print("Get CSE101:", get_course("CSE101"))
    print("Remove CSE102:", remove_course("CSE102"))
    print("Get CSE102 after removal:", get_course("CSE102"))

# Main execution
if __name__ == "__main__":
    test_courses()
    generate_docs_and_open()
```

⌵

Get CSE101: {'name': 'Data Structures', 'credits': 4}
Remove CSE102: True
Get CSE102 after removal: None
wrote __main__.html
Documentation generated and opened in browser: course.html